
VITYARTHI

YOUR LEARNING DESTINATION

PROJECT REPORT

Student Attendance Management System

Student Name	Vidhyashri Gajanan Achkulwar
Registration No.	26BAI10571
Project Type	Individual Academic Project
Technology	Python 3.x
Application Type	Console-based Attendance Management System

Submitted as part of the flipped course project evaluation.

1. Introduction

Attendance is an important part of academic administration because it helps institutions monitor participation, identify students with attendance shortages, and maintain an accurate record of classes attended. Manual attendance registers can become difficult to manage when the number of students and subjects increases. This project presents a simple Python-based Student Attendance Management System designed to organize these activities through a menu-driven console application.

The application provides student management, attendance marking, subject-wise attendance, attendance percentage calculation, defaulter identification, class statistics, dashboard information, and attendance history. The implementation uses Python functions, dictionaries, lists, loops, conditional statements, and input validation.

The project is intentionally designed as a lightweight application that is easy to understand, run, test, and extend. Data is maintained in memory during execution, so the current version does not require an external database.

2. Problem Statement

Manual attendance management requires repeated counting of present and absent records and manual calculation of attendance percentages. It can also make it difficult to quickly identify students whose attendance is below the required threshold. A simple computerized solution is therefore needed to record attendance and generate useful summaries.

The project addresses the following problems:

- Maintaining student records in an organized manner.
- Recording present and absent attendance without repeated manual calculations.
- Calculating total classes, total present, total absent, and attendance percentage.
- Managing attendance separately for different subjects.
- Identifying students with attendance below 75%.
- Providing quick class-level and student-level reports.
- Maintaining a date-based attendance history during the current program session.

3. Functional Requirements

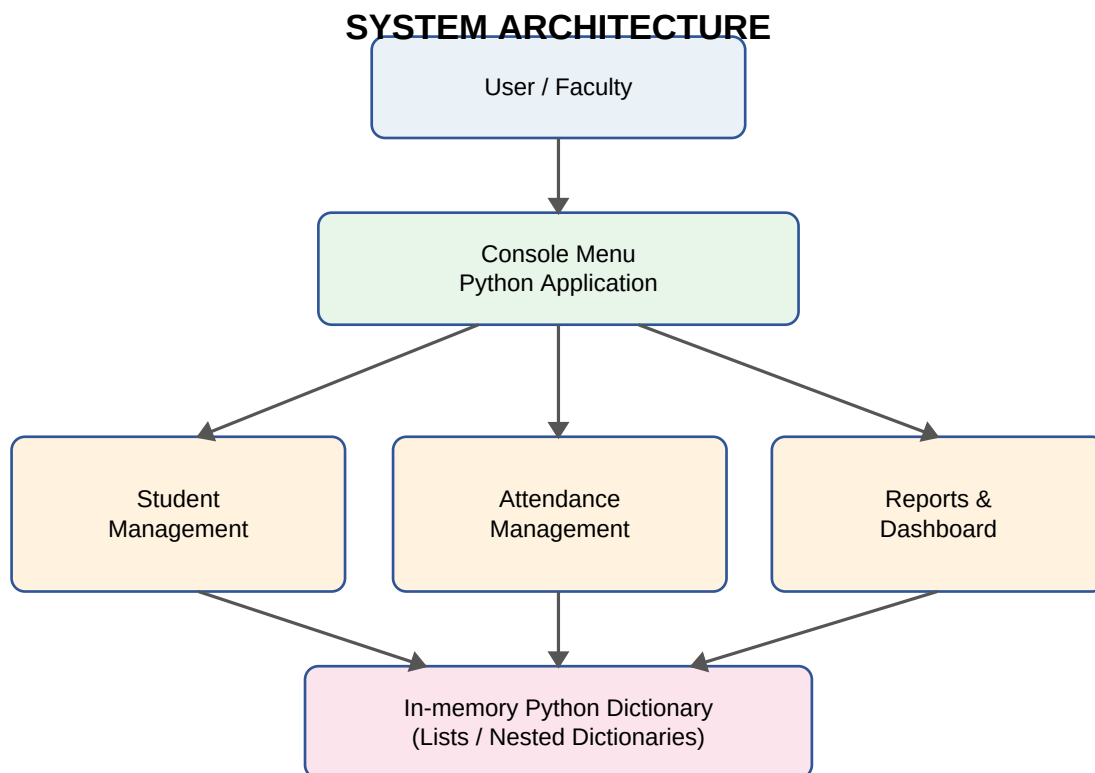
FR-01	Student Management	Add, display, edit, delete, and search student records.
FR-02	Attendance Management	Mark a student as Present or Absent.
FR-03	Attendance Calculation	Calculate total classes and attendance percentage.
FR-04	Subject Management	Add subjects for a student and maintain subject attendance.
FR-05	Reports	Generate student, subject, defaulter, and class reports.
FR-06	Dashboard	Display total students, present, absent, classes, and overall percentage.
FR-07	Attendance History	Record date, subject, and attendance status during execution.
FR-08	Validation	Reject unknown students, subjects, and invalid attendance choices.

4. Non-Functional Requirements

Usability	The menu-based interface uses clear numbered choices and simple prompts.
Reliability	The program checks whether a student or subject exists before updating records.
Maintainability	Each major operation is implemented as a separate Python function.
Error Handling	Invalid choices and missing records produce understandable messages.
Performance	In-memory dictionaries allow quick access for small and medium class sizes.
Resource Efficiency	The console application requires minimal memory and no external services.

5. System Architecture

The system follows a simple layered functional architecture. The user interacts with a console menu. The menu calls individual Python functions for student management, attendance management, subject management, reporting, and dashboard operations. The application maintains its data in an in-memory nested dictionary. This design avoids unnecessary infrastructure for a small academic project and keeps the implementation easy to understand.

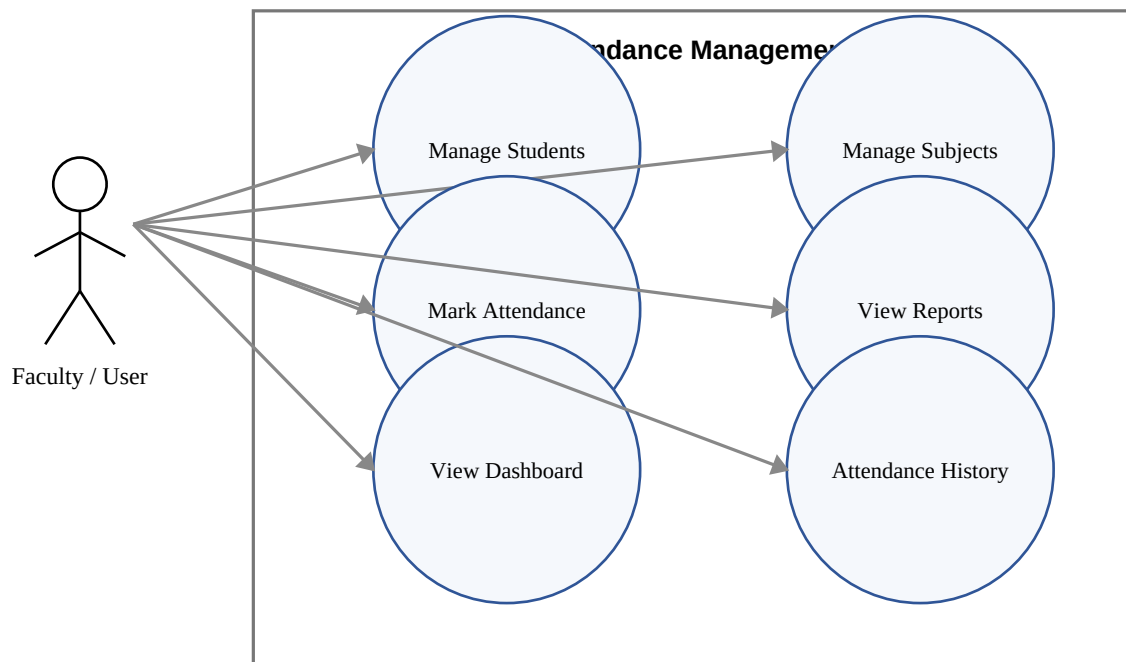


Data flow: User input → Menu → Functional module → Validation → In-memory data update → Calculation/report generation → Console output.

6. Design Diagrams

6.1 Use Case Diagram

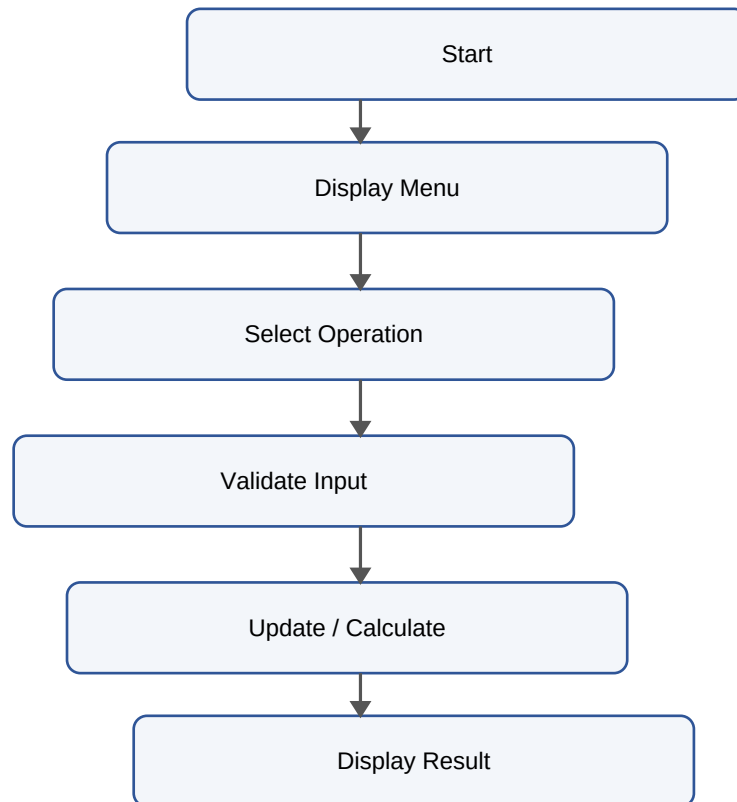
USE CASE DIAGRAM



The primary actor is the faculty/user. The actor can manage students and subjects, mark attendance, view reports, access the dashboard, and view attendance history.

6.2 Workflow Diagram

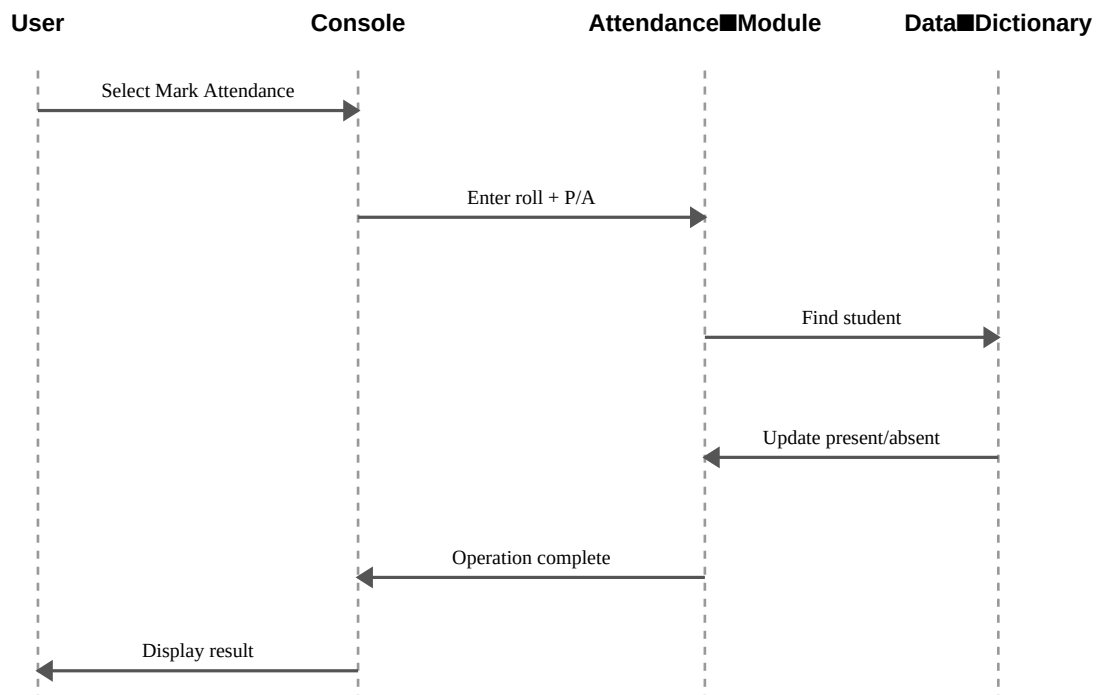
WORKFLOW DIAGRAM



The workflow begins with the menu, accepts an operation, validates the input, performs the requested calculation or update, and displays the result. The menu is then available for another operation.

6.3 Sequence Diagram

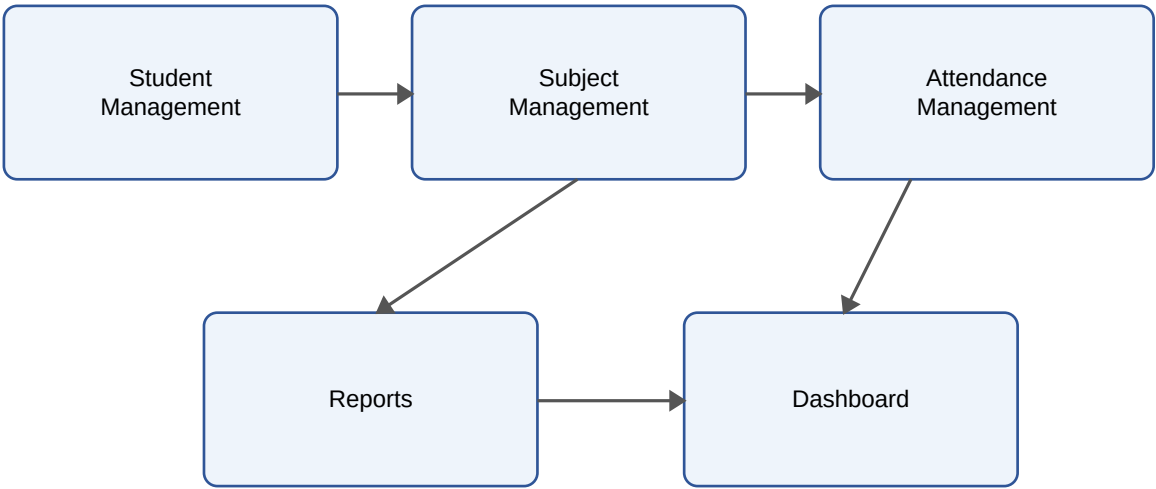
SEQUENCE DIAGRAM — MARK ATTENDANCE



For marking attendance, the user selects the attendance operation, enters a roll number and status, the application finds the student, updates the corresponding counter, and returns a confirmation.

6.4 Component Diagram

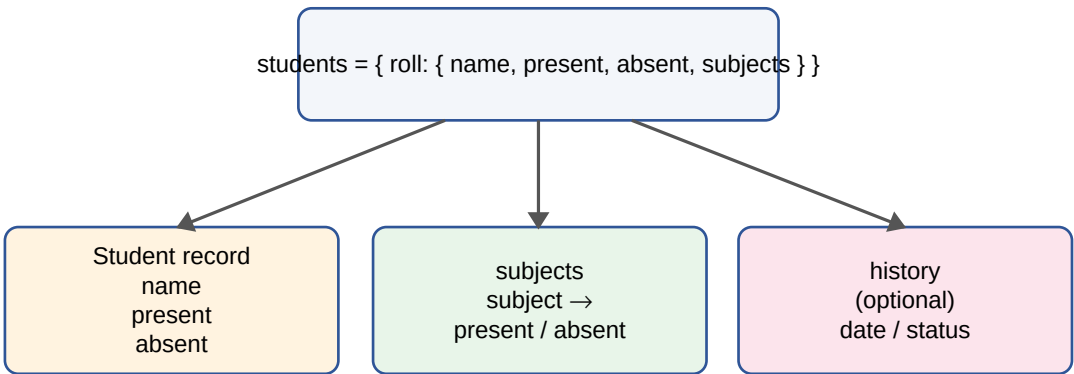
COMPONENT / MODULE DIAGRAM



The project is organized into functional components rather than a large object-oriented class hierarchy. This is suitable for the current procedural Python implementation.

6.5 Data Structure / Storage Design

IN-MEMORY DATA STRUCTURE



No external database is used in the current version. Therefore, a conventional ER diagram is not required for the implemented system. The figure above documents the actual in-memory storage structure. A future database version could convert these structures into Student, Subject, Attendance, and AttendanceHistory tables.

7. Design Decisions & Rationale

Python: Selected because it provides a simple syntax and directly supports dictionaries, lists, functions, loops, and conditional logic required for the project.

Console Interface: Chosen to keep the project focused on attendance logic rather than GUI development and to make the system easy to run in a standard Python environment.

Dictionary-based Storage: Nested dictionaries provide a straightforward way to associate a roll number with student details, attendance counters, subjects, and history.

Modular Functions: Separate functions make individual operations easier to understand, test, maintain, and explain.

75% Threshold: A 75% threshold is used for the project's eligibility/shortage logic and is clearly displayed in reports.

8. Implementation Details

8.1 Main Data Structure

```
students = {  
    "101": {  
        "name": "Student Name",  
        "present": 4,  
        "absent": 1,  
        "subjects": {  
            "Python": {  
                "present": 4,  
                "absent": 1  
            }  
        }  
    }  
}
```

8.2 Attendance Percentage

```
total = present + absent  
  
if total == 0:  
    percentage = 0  
else:  
    percentage = (present / total) * 100
```

The application calculates total classes by adding present and absent counts. When no attendance record exists, the percentage is set to zero to avoid division by zero.

8.3 Major Python Functions

- **add_student()** — creates a new student record.
- **show_students()** — displays the student list and attendance counters.
- **mark_attendance()** — updates overall present/absent counters.
- **show_attendance()** — calculates and displays an individual report.
- **show_defaulters()** — identifies students below 75%.
- **class_statistics()** — calculates class-level totals and overall attendance.
- **add_subject()** — creates a subject record for a student.
- **mark_subject_attendance()** — updates subject-specific attendance.

-
- **subject_report()** — displays subject attendance percentages.
 - **edit_student(), delete_student(), search_student()** — provide student management operations.
 - **dashboard()** — summarizes the current attendance state.
 - **add_attendance_history() and show_history()** — record and display date-based history.

9. Screenshots / Results

The following sample console results represent the expected output of the implemented features. For the final submission, screenshots from the student's own execution can be inserted below these examples if required by the evaluator.

9.1 Main Menu

```
===== ATTENDANCE SYSTEM =====
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
```

9.2 Individual Attendance Report

```
===== ATTENDANCE REPORT =====
Roll Number: 101
Student Name: Rahul
Total Classes: 5
Total Present: 4
Total Absent: 1
Attendance Percentage: 80.0 %
Status: Eligible
```

9.3 Subject Report

```
===== SUBJECT ATTENDANCE =====
Subject: Python
Present: 4
Absent: 1
Total Classes: 5
Attendance: 80.0 %
Status: Eligible
```

9.4 Dashboard

```
===== ATTENDANCE DASHBOARD =====
Total Students: 2
Total Present: 5
Total Absent: 2
Total Classes: 7
Overall Attendance: 71.43 %
```

10. Testing Approach

Testing was performed using functional/manual test cases. Each menu operation was executed with valid and invalid inputs to verify that the expected output was produced.

TC-01	Add new student	Valid roll and name	Student added successfully	Pass
TC-02	Duplicate student	Existing roll number	Duplicate rejected	Pass
TC-03	Mark Present	Existing student + P	Present counter increases	Pass
TC-04	Mark Absent	Existing student + A	Absent counter increases	Pass

TC-01	Add new student	Valid roll and name	Student added successfully	Pass
TC-05	Attendance report	Existing student	Correct total and percentage	Pass
TC-06	Add subject	Existing student + subject	Subject created	Pass
TC-07	Subject report	Subject with attendance	Correct subject percentage	Pass
TC-08	Defaulter report	Student below 75%	Student listed	Pass
TC-09	Edit/Delete/Search	Existing student	Requested operation performed	Pass
TC-10	Dashboard	Multiple records	Class totals displayed	Pass
TC-11	History	Date + subject + P/A	History displayed	Pass
TC-12	Invalid choice	Unsupported menu value	Error message shown	Pass

11. Challenges Faced

- Designing a menu that remains easy to navigate as more functions are added.
- Maintaining separate overall and subject-wise attendance counts.
- Handling invalid roll numbers and subject names without causing program failure.
- Avoiding division-by-zero errors when a student has no attendance records.
- Keeping the implementation modular while using a simple console interface.
- Managing date-based history without introducing a database dependency.

12. Learnings & Key Takeaways

- Practical use of Python functions for modular program design.
- Using dictionaries and nested dictionaries to organize related information.
- Using loops to process multiple student records.
- Using conditional statements for eligibility and validation logic.
- Designing menu-driven applications.
- Calculating and presenting meaningful statistics from raw attendance data.
- Thinking about usability, reliability, validation, and maintainability in addition to basic coding.

13. Future Enhancements

- SQLite or another database for permanent storage.
- Graphical user interface using Tkinter or a web interface.
- Faculty/student login and role-based access.
- CSV and PDF report export.
- Monthly and semester-wise attendance analytics.
- Charts showing attendance trends.
- Automatic backup and restore.
- Cloud-based storage for multi-device access.
- Automated reminders for students with low attendance.

14. References

- Python Software Foundation. Python Documentation — Python 3.x language and standard library concepts.
- Course/project guidelines provided through VITyarthi.
- Python concepts studied during the course: functions, dictionaries, lists, loops, conditional statements, and input/output.

Note: This report describes the implemented console-based version. The current application uses in-memory storage and does not claim permanent database persistence.

15. Conclusion

The Student Attendance Management System demonstrates how fundamental Python programming concepts can be combined to solve a practical academic problem. The project provides student management, attendance recording, subject-wise tracking, percentage calculation, reports, dashboard statistics, and attendance history. Its modular function-based design makes the program easy to understand and extend. Future versions can add permanent database storage, a graphical interface, authentication, and richer analytics.

```
===== RESTART: C:\Users\bhara\OneDrive\Desktop\Attendance_Project\attendance.py =====
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 1
Enter roll number: 1234
Enter student name: asdf
Student added successfully.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 2

Student List
1234 asdf Present: 0 Absent: 0
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
```

```
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 3
Enter roll number: 1234
Enter P for Present or A for Absent: p
Marked Present.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 4
Enter roll number: 1234

===== ATTENDANCE REPORT =====
Roll Number: 1234
Student Name: asdf
Total Classes: 1
Total Present: 1
Total Absent: 0
Attendance Percentage: 100.0 %
Status: Eligible
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
```

===== ATTENDANCE REPORT =====

Roll Number: 1234
Student Name: asdf
Total Classes: 1
Total Present: 1
Total Absent: 0
Attendance Percentage: 100.0 %
Status: Eligible
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 5

===== STUDENTS BELOW 75% =====

No student has attendance below 75%.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 6

===== CLASS STATISTICS =====

```
===== CLASS STATISTICS =====
Total Students: 1
Total Present: 1
Total Absent: 0
Total Attendance Records: 1
Overall Attendance: 100.0 %
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 7
Enter student roll number: 1234
Enter subject name: english
Subject added successfully.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 8
Enter roll number: 1234

Subjects:
```

Subjects:
- english
Enter subject name: english
Enter P for Present or A for Absent: p
Subject attendance marked Present.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 9
Enter roll number: 1234

===== SUBJECT ATTENDANCE =====

Subject: english
Present: 1
Absent: 0
Total Classes: 1
Attendance: 100.0 %
Status: Eligible
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard

===== SUBJECT ATTENDANCE =====

```
Subject: english
Present: 1
Absent: 0
Total Classes: 1
Attendance: 100.0 %
Status: Eligible
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 11
Enter roll number: 1234
Current name: asdf
Enter new student name: asdfg
Student details updated successfully.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 12
Enter roll number: 1234
```

```
Enter choice: 12
Enter roll number: 1234
Student deleted successfully.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 13
Enter roll number: 12345
Student not found.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 14

===== ATTENDANCE DASHBOARD =====
Total Students: 0
Total Present: 0
Total Absent: 0
Total Classes: 0
Overall Attendance: 0 %
1. Add Student
```

===== ATTENDANCE DASHBOARD =====

```
Total Students: 0
Total Present: 0
Total Absent: 0
Total Classes: 0
Overall Attendance: 0 %
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 15
Enter roll number: 1234
Student not found.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 16
Enter roll number: 1234
Student not found.
1. Add Student
2. Show Students
3. Mark Attendance
```

```
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 15
Enter roll number: 1234
Student not found.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 16
Enter roll number: 1234
Student not found.
1. Add Student
2. Show Students
3. Mark Attendance
4. Show Attendance Report
5. Show Defaulters
6. Class Statistics
7. Add Subject
8. Mark Subject Attendance
9. Subject Report
10. Exit
11. Edit Student
12. Delete Student
13. Search Student
14. Dashboard
15. Add Attendance History
16. Show Attendance History
Enter choice: 10
Program ended.
```